PostgreSQL Interview Q&A

Explained Clearly

Plain-language answers with analogies,
worked examples and interviewer tips

Edition: October 2026
Add your own real project examples to every answer.

Contents

1. How PostgreSQL Works Inside **5**

Q1. How is PostgreSQL built? Explain its process architecture. . . . 5

Q2. What is MVCC and why does PostgreSQL need VACUUM because of it? . . . 5

Q3. What is WAL (Write-Ahead Log) and why does it exist? . . . 6

Q4. What is a checkpoint, and what goes wrong when checkpoints are mis-tuned? . . . 6

Q5. Walk me through what happens when you run an UPDATE and COMMIT. . . . 7

2. VACUUM, Bloat and Wraparound **8**

Q6. What does VACUUM do, and what is bloat? . . . 8

Q7. How does autovacuum decide when to run, and how do you tune it? Show a calculation. . . . 8

Q8. Autovacuum is running, but the table is still bloated. What could be blocking it? . . . 9

Q9. What is transaction ID wraparound, and how do you prevent it? . . . 9

Q10. VACUUM vs VACUUM FULL vs pg_repack: when do you use each? . . . 10

3. Indexes and Query Tuning **11**

Q11. What index types does PostgreSQL have, and when do you use each? . . . 11

Q12. How do you read EXPLAIN ANALYZE? Show me with an example. . . . 11

Q13. The table has an index, but PostgreSQL still does a sequential scan. Why? . . . 12

Q14. A query is slow in production. Walk me through your approach. . . . 12

Q15. How do you create an index on a busy table without causing an outage? Explain partial, covering and expression indexes. . . . 13

Q16. What are planner statistics, and what is an extended statistic? . . . 13

4. Transactions, Isolation and Locking **14**

Q17. Explain the PostgreSQL transaction isolation levels with an example. . . . 14

Q18. Why can a quick ALTER TABLE freeze an entire application? . . . 14

Q19. What is a deadlock and how do you prevent it? . . . 15

Q20. How would you build a safe job queue in PostgreSQL? . . . 15

5. Replication, Backup and High Availability **16**

Q21. What is the difference between physical (streaming) and logical replication? . . . 16

Q22. Synchronous vs asynchronous replication: what is the trade-off? . . . 16

Q23. What are replication slots, and how can they fill the disk? . . . 17

Q24. How do backups and point-in-time recovery (PITR) work in PostgreSQL? . . . 17

Q25. How do you upgrade a PostgreSQL major version with minimal downtime? . . . 18

6. Memory, Configuration and Connections **19**

Q26. Which memory parameters matter, and what is the danger of work_mem? . . . 19

Q27. Why do you need connection pooling, and how does PgBouncer work? . . . 19

7. Amazon RDS and Aurora PostgreSQL **20**

Q28. How is RDS/Aurora PostgreSQL different from running PostgreSQL yourself? 20
Q29. How is Aurora PostgreSQL different from RDS for PostgreSQL? 20
Q30. How do you troubleshoot a performance problem on Aurora/RDS PostgreSQL? 20

8. Scenario Questions (Model Answers) 22

Q31. Scenario: CPU on the primary is at 95%. What do you do? 22
Q32. Scenario: the database disk is filling quickly. Where do you look? 22
Q33. Scenario: the application became slow right after a release. How do you find out why? 22

9. Glossary in Plain English 23

How to use this guide

This edition explains every answer clearly. Each question follows the same pattern so you can learn it and repeat it in an interview:

Part	What it gives you
SHORT ANSWER	One or two sentences you can say immediately. Interviewers like a clear opening.
Explanation	The reasoning in plain language: what happens inside PostgreSQL and why.
THINK OF IT LIKE THIS	A real-world analogy that makes the idea easy to remember and to explain.
Example	A worked example with SQL, commands or a timeline. Output values are illustrative.
INTERVIEW TIP	What the interviewer is really checking and what they may ask next.

The 3-layer answer method. For any question, answer in three layers: (1) **what it is**, (2) **why it matters / what goes wrong**, (3) **a real example from your work**. Practise saying layer 3 for the topics you have hands-on experience with.

Version note. Defaults and features vary by PostgreSQL major version. Say which version you mean, and offer to confirm in the official documentation.

1. How PostgreSQL Works Inside

Q1. How is PostgreSQL built? Explain its process architecture.

SHORT ANSWER (say this first)
PostgreSQL starts one parent process (the postmaster). It creates one separate backend process for every client connection, and runs a few background processes that do housekeeping such as writing data to disk and cleaning old rows.

Explanation. When your application connects, the postmaster accepts the connection and forks a new operating-system process just for that session. All processes share one block of memory called **shared memory**, which contains the main cache (`shared_buffers`), the WAL buffers and the lock tables. Because every connection is a real OS process that uses memory, very large numbers of connections become expensive. This is the reason connection pooling is standard practice.

THINK OF IT LIKE THIS
A restaurant. The postmaster is the host at the door. Each guest gets a dedicated waiter (a backend process). Behind the scenes the kitchen staff (background processes) prepare food, clean tables and keep records, so the waiters never have to.

Background process	Its job in plain words
Checkpointer	Periodically writes all changed pages from memory to the data files so crash recovery stays short.
Background writer	Gradually writes some dirty pages so backends rarely have to wait for disk.
WAL writer	Flushes the write-ahead log from memory to disk.
Autovacuum launcher / workers	Clean up dead rows, refresh statistics, and freeze old transaction IDs.
Archiver	Copies completed WAL files to backup storage (for point-in-time recovery).
WAL sender / receiver	Stream WAL between a primary and its replicas.

Example. See the processes yourself:

```
SELECT pid, backend_type, state FROM pg_stat_activity ORDER BY backend_type;
```

INTERVIEW TIP / LIKELY FOLLOW-UP
The practical consequence is the point they want: "one process per connection, so I use a pooler like PgBouncer or RDS Proxy and keep `max_connections` modest." Follow-up: "How many connections should you allow?"

Q2. What is MVCC and why does PostgreSQL need VACUUM because of it?

SHORT ANSWER (say this first)
MVCC (Multi-Version Concurrency Control) keeps several versions of a row so readers and writers do not block each other. The price is that old versions pile up as dead rows, and VACUUM must clean them.

Explanation. Each row version has two hidden fields: **xmin** (the transaction that created it) and **xmax** (the transaction that deleted or replaced it). When a query starts, it takes a **snapshot** of which transactions are committed. It then sees only the row versions that were valid for that snapshot. An `UPDATE` does not change the row in place: it writes a **new version** and marks the old one as ended. A `DELETE` just marks the row as ended. The old versions remain until no running transaction can possibly need them, and then VACUUM reclaims their space.

THINK OF IT LIKE THIS
A library that never erases a page. When a fact changes, it adds a new page with the new fact and stamps the old page "replaced at 10:05". Readers who started at 10:00 keep reading the old page; new readers get the new page. Later a librarian (VACUUM) throws away old pages nobody needs.

Example. Watch a row get a new version:

```
CREATE TABLE acct (id int PRIMARY KEY, bal int);
INSERT INTO acct VALUES (1, 100);
SELECT xmin, xmax, ctid, * FROM acct;      -- xmin=750  ctid=(0,1)  bal=100
UPDATE acct SET bal = 90 WHERE id = 1;
SELECT xmin, xmax, ctid, * FROM acct;      -- xmin=751  ctid=(0,2)  bal=90  (a NEW physical row)
-- The old version at ctid (0,1) is now a dead tuple until VACUUM removes it.
```

INTERVIEW TIP / LIKELY FOLLOW-UP
Connect MVCC to its two consequences: **bloat** (dead rows) and **transaction ID wraparound** (IDs are 32-bit). Follow-ups: "What is a HOT update?" and "Why do long transactions hurt?" (they keep old versions alive).

Q3. What is WAL (Write-Ahead Log) and why does it exist?

SHORT ANSWER (say this first)
WAL records every change in a sequential log and flushes it to disk before the change is considered committed. It makes the database crash-safe and powers replication and point-in-time recovery.

Explanation. Writing changed data pages directly to their places in data files would be slow (random I/O) and unsafe if the server crashed halfway. Instead PostgreSQL changes the page in memory, appends a small description of the change to the WAL (fast, sequential), and on `COMMIT` forces the WAL to disk. The data pages themselves are written later by the checkpointer. After a crash, PostgreSQL replays the WAL from the last checkpoint to rebuild any change that had not reached the data files.

THINK OF IT LIKE THIS
A shop ledger. The cashier writes every sale in the ledger first (quick, in order). The shelves (data files) are updated later. If the power fails, the manager reads the ledger and fixes the shelves.

- **Crash recovery:** replay WAL after a crash.
- **Replication:** standbys receive and replay the same WAL stream.
- **PITR:** keep archived WAL and replay it up to a chosen time.
- `synchronous_commit` decides whether `COMMIT` waits for the WAL flush (and optionally for a standby). Turning it off can lose the last few commits in a crash, but never corrupts the database.

INTERVIEW TIP / LIKELY FOLLOW-UP
Mention that WAL volume is a performance and storage factor: bulk updates and unlogged index rebuilds generate a lot of WAL, which also lags replicas.

Q4. What is a checkpoint, and what goes wrong when checkpoints are mis-tuned?

SHORT ANSWER (say this first)
A checkpoint writes all modified pages from memory to the data files and records a safe restart point. It limits how much WAL must be replayed after a crash.

Explanation. Between checkpoints, changes live in memory and in the WAL. When a checkpoint happens, PostgreSQL flushes the dirty pages, so WAL before that point is no longer needed for crash recovery. If checkpoints are too frequent, you get repeated I/O bursts and extra full-page writes. If they are too far apart, crash recovery takes longer and more WAL is kept. Main settings: `checkpoint_timeout` (default 5 minutes), `max_wal_size` and `checkpoint_completion_target` (spreads the writing out).

How to spot a problem: log messages such as "checkpoints are occurring too frequently", or many *requested* checkpoints compared to *timed* ones in the checkpoint statistics view. **Fix:** raise `max_wal_size` so most checkpoints are triggered by time, not by WAL volume.

INTERVIEW TIP / LIKELY FOLLOW-UP
Note that on Aurora PostgreSQL the storage layer handles durability differently, so classic checkpoint tuning is not the main lever there.

Q5. Walk me through what happens when you run an UPDATE and COMMIT.

SHORT ANSWER (say this first)
PostgreSQL writes a new row version in memory, logs the change to WAL, flushes the WAL on commit, and writes the data page to disk later. The old row version stays until VACUUM removes it.

1. The backend takes a snapshot and finds the row using an index or scan.
2. It takes a row lock, creates a **new tuple version** in a page in shared buffers, and sets `xmax` on the old version.
3. It writes a **WAL record** describing the change (into WAL buffers).
4. On `COMMIT`, the WAL is **flushed to disk** (and sent to a synchronous standby if configured). Now the client receives "COMMIT".
5. Later, the **checkpointer/background writer** writes the changed page to the data file.
6. Later still, **autovacuum** removes the old version once no snapshot can see it, and updates statistics.

INTERVIEW TIP / LIKELY FOLLOW-UP
This end-to-end story combines MVCC, WAL, checkpoints and vacuum. It is one of the best ways to show senior-level understanding in two minutes.

2. VACUUM, Bloat and Wraparound

Q6. What does VACUUM do, and what is bloat?

SHORT ANSWER (say this first)
VACUUM finds dead row versions and marks their space reusable, updates the visibility map, and freezes old rows. Bloat is wasted space inside tables and indexes caused by dead rows that were not cleaned or reused.

Explanation. After many updates and deletes, a table contains live rows and dead rows. Plain VACUUM does not shrink the file in most cases; it lets later inserts and updates **reuse** the space. If vacuum cannot keep up, the table keeps growing, queries scan more pages, indexes grow, cache is wasted and disk is consumed. That wasted space is **bloat**.

THINK OF IT LIKE THIS
A parking lot where cars (rows) leave but the empty spots are never marked "free". New cars keep being parked in new spots and the lot keeps expanding. VACUUM updates the board so empty spots can be reused.

Example. Check how dirty a table is:

```
SELECT relname, n_live_tup, n_dead_tup,
       round(100.0 * n_dead_tup / nullif(n_live_tup + n_dead_tup, 0), 1) AS dead_pct,
       last_autovacuum
FROM pg_stat_user_tables ORDER BY n_dead_tup DESC LIMIT 5;
-- orders | 9,800,000 | 2,400,000 | 19.7 | 2026-10-27 03:12   <- vacuum is falling behind
```

INTERVIEW TIP / LIKELY FOLLOW-UP
Show that you treat bloat as a symptom: the cause is usually autovacuum tuning or something blocking cleanup (Q8).

Q7. How does autovacuum decide when to run, and how do you tune it? Show a calculation.

SHORT ANSWER (say this first)
Autovacuum vacuums a table when dead rows exceed `threshold + scale_factor x number_of_rows`. The default scale factor (20%) is too lazy for big tables, so you set smaller values per table.

Explanation. The defaults are `autovacuum_vacuum_threshold = 50` and `autovacuum_vacuum_scale_factor = 0.2`. For analyze they are 50 and 0.1. Since PostgreSQL 13, inserts also trigger vacuum (insert threshold 1000, scale 0.2) so insert-only tables still get frozen and have the visibility map set.

Table size	Dead rows needed to trigger vacuum (default 0.2)	With scale_factor = 0.01
10,000 rows	50 + 0.2 x 10,000 = 2,050	50 + 0.01 x 10,000 = 150
10,000,000 rows	50 + 0.2 x 10,000,000 = **2,000,050**	50 + 0.01 x 10,000,000 = **100,050**
500,000,000 rows	**about 100 million** dead rows	about 5 million

So on a 10-million-row table, the default waits for two million dead rows. That is far too late for a busy table.

```
ALTER TABLE public.orders SET (
  autovacuum_vacuum_scale_factor  = 0.01,
  autovacuum_analyze_scale_factor = 0.005,
  autovacuum_vacuum_cost_limit    = 2000);   -- let vacuum work faster on this table
```

- Globally: raise `autovacuum_max_workers` for many big tables, increase the cost limit so vacuum can keep up, and give it enough `autovacuum_work_mem`.
- Log what it does with `log_autovacuum_min_duration`.

INTERVIEW TIP / LIKELY FOLLOW-UP
Showing the arithmetic (2,000,050) is impressive and proves you understand the formula, not just the parameter names.

Q8. Autovacuum is running, but the table is still bloated. What could be blocking it?

SHORT ANSWER (say this first)
Something is holding back the "cleanup horizon": an old open transaction, a forgotten prepared transaction, a replication slot, or a standby with `hot_standby_feedback` running a long query.

Explanation. VACUUM may only remove a dead row if **no current or future snapshot** could still need it. The oldest snapshot in the system sets the limit. If one transaction has been open for three days, nothing newer than its snapshot can be cleaned, no matter how often autovacuum runs.

Check these four things:

```
-- 1) Oldest transactions / idle in transaction
SELECT pid, usename, state, now()-xact_start AS xact_age, age(backend_xmin) AS xmin_age
FROM pg_stat_activity WHERE backend_xmin IS NOT NULL ORDER BY age(backend_xmin) DESC LIMIT 5;
-- 2) Orphaned prepared transactions
SELECT gid, prepared FROM pg_prepared_xacts;
-- 3) Replication slots holding the horizon back
SELECT slot_name, active, xmin, catalog_xmin FROM pg_replication_slots;
-- 4) Standbys: long queries + hot_standby_feedback = primary cannot clean up
```

Fix: terminate or fix the offender (`pg_terminate_backend`, `ROLLBACK PREPARED`, `pg_drop_replication_slot`), then set `idle_in_transaction_session_timeout` so it cannot happen again.

INTERVIEW TIP / LIKELY FOLLOW-UP
This is a classic senior question. The key phrase is "the oldest open snapshot holds back cleanup".

Q9. What is transaction ID wraparound, and how do you prevent it?

SHORT ANSWER (say this first)
Transaction IDs are 32-bit numbers that wrap around. Old rows must be "frozen" by vacuum before the counter laps them, otherwise PostgreSQL stops accepting writes to protect data.

Explanation. PostgreSQL decides whether a row is in the past or the future by comparing transaction IDs in a circle. Roughly two billion IDs are "past" and two billion are "future". If a row stays unfrozen too long, it could suddenly look like it belongs to the future and vanish. **Freezing** replaces the ID of an old row with a special marker meaning "visible to everyone". Autovacuum does this automatically; when a table is older than `autovacuum_freeze_max_age` (default 200 million) it starts an **aggressive** freeze vacuum even if autovacuum is otherwise off.

THINK OF IT LIKE THIS
A clock with only 12 numbers. If you want to say "this happened 11 hours ago" you can, but after 12 hours the clock cannot tell the past from the future. Freezing is like writing the full date on old photos so the clock face no longer matters.

Stage	Meaning	What to do
age < 200 million	Healthy	Nothing. Keep monitoring.
200 million - 500 million	Aggressive vacuums running	Check why normal vacuum did not freeze; tune autovacuum.
> 1 billion	Danger zone	Find blockers, freeze oldest tables manually, raise vacuum capacity.
Within a few million of 2.1 billion	PostgreSQL refuses new write transactions	Emergency: single-user vacuum / AWS Support if managed.

```
SELECT datname, age(datfrozenxid) FROM pg_database ORDER BY 2 DESC;                -- database
    level
SELECT relname, age(relfrozenxid) FROM pg_class WHERE relkind='r' ORDER BY 2 DESC LIMIT 5;   --
    table level
```

```
VACUUM (FREEZE, VERBOSE) public.orders;                                    -- fix the oldest
    tables first
```

INTERVIEW TIP / LIKELY FOLLOW-UP
Mention monitoring: on RDS/Aurora the CloudWatch metric `MaximumUsedTransactionIDs`. Alert at 500 million, not at the last minute.

Q10. VACUUM vs VACUUM FULL vs pg_repack: when do you use each?

SHORT ANSWER (say this first)
Use plain VACUUM routinely (autovacuum). Use pg_repack to shrink a bloated table online. Use VACUUM FULL only in a maintenance window because it locks the table.

Option	Lock	What it does	Use when
VACUUM	Does not block reads/writes	Marks space reusable, freezes, updates visibility map	Always, via autovacuum.
VACUUM FULL	ACCESS EXCLUSIVE (blocks everything)	Rewrites the table into a new file and returns space to the OS	Rare, with downtime, and enough free disk.
pg_repack	Brief locks only	Rebuilds the table/indexes online in the background	Large bloated table in production.
REINDEX CONCURRENTLY	No long blocking	Rebuilds a bloated index online	Index bloat (PostgreSQL 12+).

INTERVIEW TIP / LIKELY FOLLOW-UP
A strong answer ends with the root cause fix: "I would rebuild once, then tune autovacuum so it does not return."

3. Indexes and Query Tuning

Q11. What index types does PostgreSQL have, and when do you use each?

SHORT ANSWER (say this first)
B-tree is the default and covers most cases. GIN is for JSONB, arrays and full-text. GiST is for geometry and ranges. BRIN is for huge ordered tables. Hash is rarely needed.

Index	Good for	Everyday example
B-tree	Equality, ranges, sorting, prefix match	`WHERE id = 5, ORDER BY created_at`
GIN	Many keys inside one value: JSONB, arrays, text search, trigram	`WHERE data @> '{"type":"login"}'`
GiST	Geometric data, ranges, nearest-neighbour	PostGIS maps, overlapping time ranges
SP-GiST	Space-partitioned data (prefixes, quadtrees)	IP ranges, phone-prefix lookups
BRIN	Very large tables whose values follow physical order	Time-series log table by timestamp
Hash	Equality only	Rarely better than B-tree

THINK OF IT LIKE THIS
A B-tree is a phone book (sorted, great for names and ranges). A GIN index is the index at the back of a textbook: for each word, a list of the pages where it appears. A BRIN index is a note on each chapter saying "dates in this chapter run from March to May".

INTERVIEW TIP / LIKELY FOLLOW-UP
Trade-off thinking earns points: every index speeds reads but slows writes, uses disk and creates vacuum work.

Q12. How do you read EXPLAIN ANALYZE? Show me with an example.

SHORT ANSWER (say this first)
`EXPLAIN` shows the plan the planner chose. Adding `ANALYZE` runs the query and shows actual time and rows. You compare estimates with actuals and find the slow node.

Explanation. Read the plan from the innermost (most indented) node outward. Each node shows `cost` (planner estimate), `rows` (estimated), then in `actual` the real time and rows. A big gap between estimated and actual rows means bad statistics. Add `BUFFERS` to see how many pages came from cache (`hit`) or storage (`read`).

Before: a sequential scan.

```
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM orders WHERE customer_id = 42;

Seq Scan on orders  (cost=0.00..18334.00 rows=5 width=64) (actual time=0.015..95.2 rows=7 loops=1)
  Filter: (customer_id = 42)
  Rows Removed by Filter: 999993          <- read a million rows to return 7
  Buffers: shared hit=5334
Execution Time: 95.3 ms
```

Fix: create the index (online) and run it again.

```
CREATE INDEX CONCURRENTLY idx_orders_customer ON orders (customer_id);

Index Scan using idx_orders_customer on orders (cost=0.42..8.50 rows=5 width=64) (actual
    time=0.03..0.04 rows=7 loops=1)
  Index Cond: (customer_id = 42)
Execution Time: 0.06 ms                    <- about 1,500 times faster
```

You see in the plan	It means	Typical fix
Seq Scan + large "Rows Removed by Filter"	No useful index	Create an index on the filter columns

You see in the plan	It means	Typical fix
Estimated rows 10, actual rows 500,000	Bad statistics	ANALYZE, raise statistics target, extended statistics
`Sort Method: external merge Disk`	Sort spilled to disk (work_mem too low)	Increase work_mem for that query/role, or add an index
`Hash Batches: 8`	Hash join spilled	Same: more work_mem, or filter earlier
Heap Fetches high in Index Only Scan	Visibility map is stale	Vacuum the table more often

INTERVIEW TIP / LIKELY FOLLOW-UP
Always say "on a realistic copy" or "carefully, because ANALYZE really runs the query" (wrap data-changing statements in a transaction and roll back).

Q13. The table has an index, but PostgreSQL still does a sequential scan. Why?

SHORT ANSWER (say this first)
Because the planner thinks scanning everything is cheaper, or because the index cannot be used by that query.

- **Large fraction of rows:** if the query returns 30% of the table, reading it sequentially beats jumping around with an index.
- **Stale statistics:** the planner believes the table is tiny or the filter is unselective. Run `ANALYZE`.
- **Query cannot use the index:** a function on the column (`WHERE lower(email)=...` without an expression index), a type mismatch (comparing a text column to a number), or a leading wildcard (`LIKE '%abc'`).
- **Cost settings:** `random_page_cost` defaults to 4 (tuned for spinning disks). On SSD or cloud storage, 1.1-1.5 is common.
- **Tiny table:** scanning one page is faster than using an index.

Example fix for a function on a column:

```
CREATE INDEX idx_users_email_lower ON users (lower(email));      -- now WHERE lower(email) =
    'a@b.com' can use it
```

INTERVIEW TIP / LIKELY FOLLOW-UP
Do not say "force the index." Say you would find out why the planner prefers a scan, and fix statistics, the query or the cost settings.

Q14. A query is slow in production. Walk me through your approach.

SHORT ANSWER (say this first)
Find the exact slow statement, look at its plan, fix the biggest problem, and measure again.

1. **Identify:** use `pg_stat_statements` (top total time, top mean time), the slow query log, or Performance Insights on RDS/Aurora.
2. **Reproduce safely:** run `EXPLAIN (ANALYZE, BUFFERS)` on a clone or replica with realistic data and the real parameter values.
3. **Find the bottleneck node:** seq scan on a big table, bad row estimate, sort/hash spill, nested loop with huge inner side.
4. **Check the basics:** statistics freshness (`last_analyze`), table/index bloat, missing or unused indexes.
5. **Fix one thing:** index, query rewrite, `ANALYZE`, `work_mem` for the role, extended statistics, or partitioning.
6. **Verify and record:** compare time, buffers and rows before and after, and watch the effect on writes.

```
SELECT queryid, calls, round(total_exec_time::numeric) AS total_ms, round(mean_exec_time::numeric,2)
    AS mean_ms,
       left(query, 80) AS query
FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 10;
```

INTERVIEW TIP / LIKELY FOLLOW-UP
Mention the difference between *one slow query* (mean time) and *death by a thousand cuts* (high calls x small time). Total time finds both.

Q15. How do you create an index on a busy table without causing an outage? Explain partial, covering and expression indexes.

SHORT ANSWER (say this first)
Use `CREATE INDEX CONCURRENTLY`, which builds the index without blocking writes. Use partial, covering or expression indexes to make indexes smaller and more useful.

Explanation. A normal `CREATE INDEX` blocks writes to the table while it builds. `CONCURRENTLY` takes longer and cannot run inside a transaction block, but normal reads and writes continue. If it fails, it leaves an **INVALID** index that you must drop and retry.

```
SET lock_timeout = '5s';
CREATE INDEX CONCURRENTLY idx_orders_open ON orders (customer_id) WHERE status = 'OPEN';   --
    partial
CREATE INDEX CONCURRENTLY idx_orders_cov  ON orders (customer_id) INCLUDE (total, created_at); --
    covering
CREATE INDEX CONCURRENTLY idx_users_lower ON users (lower(email));                          --
    expression
SELECT indexrelid::regclass FROM pg_index WHERE NOT indisvalid;                             -- find
    failed builds
```

Type	Why it helps
Partial	Indexes only the rows you query (for example open orders). Much smaller and faster to maintain.
Covering (`INCLUDE`)	Stores extra columns so the query can be answered from the index alone (index-only scan).
Expression	Matches queries that filter on a function result, such as `lower(email)`.

INTERVIEW TIP / LIKELY FOLLOW-UP
A follow-up is "How do you find indexes to drop?": unused indexes in `pg_stat_user_indexes` (idx_scan = 0 over a full business cycle), plus duplicates.

Q16. What are planner statistics, and what is an extended statistic?

SHORT ANSWER (say this first)
Statistics are the planner's knowledge about your data (row counts, value distribution). Bad statistics cause bad plans. Extended statistics teach it about relationships between columns.

Explanation. `ANALYZE` samples each table and stores distribution data in `pg_stats`. The planner uses it to estimate how many rows each filter returns, and from that it chooses scans and join methods. By default it assumes columns are **independent**. If `city = 'Paris'` and `country = 'France'` are strongly related, multiplying their selectivities gives a far too small estimate, so the planner chooses a poor plan.

```
ANALYZE public.orders;                                                          -- refresh stats now
ALTER TABLE public.orders ALTER COLUMN customer_id SET STATISTICS 500;          -- finer histogram
    (default 100)
CREATE STATISTICS orders_city_country (dependencies) ON city, country FROM public.orders;
ANALYZE public.orders;                                                          -- needed after
    creating it
```

INTERVIEW TIP / LIKELY FOLLOW-UP
Good signal for them: "I check estimated vs actual rows first. If they differ by 10x or more, I look at statistics before touching indexes."

4. Transactions, Isolation and Locking

Q17. Explain the PostgreSQL transaction isolation levels with an example.

SHORT ANSWER (say this first)
Isolation levels define what a transaction can see while others are changing data. PostgreSQL defaults to Read Committed; Repeatable Read gives one snapshot per transaction; Serializable behaves as if transactions ran one at a time.

Explanation. The difference is **when the snapshot is taken**. In Read Committed, every statement gets a fresh snapshot. In Repeatable Read, the snapshot is taken at the first statement and kept until the end. Serializable adds conflict detection and may abort a transaction, so the application must retry on error `40001`. PostgreSQL has no dirty reads: Read Uncommitted behaves like Read Committed.

Example. Balance is 100. Session B updates it to 90 and commits while Session A is in the middle of a transaction:

Time	Session A	Session B
1	`BEGIN; SELECT bal FROM acct WHERE id=1;` returns **100**	
2		`UPDATE acct SET bal=90 WHERE id=1; COMMIT;`
3	`SELECT bal ...` again	
Result	Read Committed: **90** (non-repeatable read). Repeatable Read: **100** (same snapshot).	

THINK OF IT LIKE THIS
Read Committed is like refreshing a web page each time you look: you always see the latest. Repeatable Read is like looking at a photo taken when you started: it does not change while you work.

INTERVIEW TIP / LIKELY FOLLOW-UP
Say that higher isolation is not free: Repeatable Read and Serializable can raise serialization errors, so the application needs retry logic.

Q18. Why can a quick ALTER TABLE freeze an entire application?

SHORT ANSWER (say this first)
DDL needs a strong lock. If it has to wait behind a long transaction, every later query on that table waits behind the DDL, creating a pile-up.

Explanation. Most `ALTER TABLE` forms take an **ACCESS EXCLUSIVE** lock, which conflicts with every other lock including plain `SELECT`. Locks are granted in queue order. So one old open transaction plus one waiting DDL can block all new traffic to that table, even though the DDL itself would finish in milliseconds.

Time	What happens	Effect
T1	A report starts a transaction and reads `orders`. It stays open for 10 minutes.	Holds a weak lock on orders
T2	You run `ALTER TABLE orders ADD COLUMN x int;`	Needs ACCESS EXCLUSIVE, **waits** for T1
T3	Application runs normal SELECT/UPDATE on orders	Queued **behind** the ALTER: application hangs
T4	T1 finally ends; ALTER runs in 5 ms	Queue drains, but you had a 10-minute outage

Safe pattern:

```
SET lock_timeout = '3s';                        -- give up quickly instead of blocking everyone;
    retry later
ALTER TABLE orders ADD COLUMN note text;        -- constant/no default is instant (PostgreSQL 11+)
```

```
ALTER TABLE orders ADD CONSTRAINT fk_c FOREIGN KEY (customer_id) REFERENCES customers(id) NOT VALID;
ALTER TABLE orders VALIDATE CONSTRAINT fk_c;    -- validation uses a weaker lock
```

Finding the blocker:

```
SELECT blocked.pid AS blocked, blocker.pid AS blocker, blocker.state, now()-blocker.xact_start AS
    blocker_age,
       left(blocker.query,60) AS blocker_query
FROM pg_stat_activity blocked
JOIN LATERAL unnest(pg_blocking_pids(blocked.pid)) b(pid) ON true
JOIN pg_stat_activity blocker ON blocker.pid = b.pid;
SELECT pg_cancel_backend(<pid>);       -- try this first
SELECT pg_terminate_backend(<pid>);    -- then this (rolls the session back)
```

INTERVIEW TIP / LIKELY FOLLOW-UP
"`lock_timeout` plus retry" is the phrase they want to hear for production DDL.

Q19. What is a deadlock and how do you prevent it?

SHORT ANSWER (say this first)
A deadlock is two transactions each waiting for a lock the other holds. PostgreSQL detects it and aborts one of them.

Step	Session A	Session B
1	`UPDATE acct SET bal=bal-10 WHERE id=1;` (locks row 1)	`UPDATE acct SET bal=bal-10 WHERE id=2;` (locks row 2)
2	`UPDATE acct SET bal=bal+10 WHERE id=2;` waits for B	
3		`UPDATE acct SET bal=bal+10 WHERE id=1;` waits for A: **deadlock**
4	After `deadlock_timeout` (1 s) PostgreSQL aborts one session: `ERROR: deadlock detected` (code 40P01)	

- **Prevent:** always lock rows and tables in the same order (for example lowest id first), keep transactions short, never wait for user input inside a transaction.
- **Handle:** the application should retry the aborted transaction.
- **Diagnose:** enable `log_lock_waits`; the log shows both statements and the processes.

INTERVIEW TIP / LIKELY FOLLOW-UP
Mention the cause is usually application logic (inconsistent lock order), not the database.

Q20. How would you build a safe job queue in PostgreSQL?

SHORT ANSWER (say this first)
Use `SELECT ... FOR UPDATE SKIP LOCKED`, so many workers can each take different rows without waiting for each other.

```
WITH next AS (
  SELECT id FROM jobs WHERE status = 'ready' ORDER BY id LIMIT 10
  FOR UPDATE SKIP LOCKED)
UPDATE jobs j SET status = 'running', started_at = now()
FROM next WHERE j.id = next.id RETURNING j.*;
```

Explanation. Without `SKIP LOCKED`, the second worker would wait for the first worker's locked rows. With it, locked rows are skipped and the worker simply takes the next available ones. Keep the work transaction short, and handle workers that crash by using a timeout column and a recovery job. Advisory locks are an alternative for application-level mutual exclusion.

INTERVIEW TIP / LIKELY FOLLOW-UP
Also mention the dead-tuple cost: queue tables update/delete heavily, so tune autovacuum aggressively on them.

5. Replication, Backup and High Availability

Q21. What is the difference between physical (streaming) and logical replication?

SHORT ANSWER (say this first)
Physical replication copies the WAL byte for byte to build an identical standby. Logical replication sends decoded row changes for chosen tables, so the target can be different.

THINK OF IT LIKE THIS
Physical replication is photocopying the whole notebook page by page. Logical replication is sending a list of changes: "row 5 in table X now has this value", only for the tables you subscribe to.

	Physical	Logical
Copies	The whole cluster	Selected tables
Target	Same major version and architecture; standby is read-only	Can be a different major version, writable, with extra tables or indexes
Main uses	High availability, read replicas, DR	Zero-downtime major upgrades, migrations, data distribution, CDC
Gotchas	Cannot pick tables; replica conflicts with long queries	No DDL replication; sequences and large objects need handling; tables need a primary key or replica identity

```
-- Logical replication in two commands
CREATE PUBLICATION pub_app FOR TABLE app.orders, app.customers;                    -- on the
    source
CREATE SUBSCRIPTION sub_app CONNECTION 'host=src dbname=appdb user=repl' PUBLICATION pub_app; -- on
    the target
```

INTERVIEW TIP / LIKELY FOLLOW-UP
Know that wal_level must be `logical` (on Aurora/RDS set `rds.logical_replication = 1` and reboot).

Q22. Synchronous vs asynchronous replication: what is the trade-off?

SHORT ANSWER (say this first)
Asynchronous is faster but can lose recent commits if the primary fails. Synchronous waits for a standby so no committed data is lost, but adds latency and can block writes if the standby is down.

THINK OF IT LIKE THIS
Asynchronous is dropping a letter in the post box and walking away. Synchronous is registered mail where you wait for the receiver's signature before leaving.

- **RPO** (data loss): async = possibly seconds of data; sync = zero.
- **Latency:** sync commits wait for the standby's WAL flush (or replay with `remote_apply`).
- **Availability:** with one sync standby, a standby outage can stall writes. Use quorum (`ANY 1 (s1, s2)`) so another standby can satisfy the commit.

INTERVIEW TIP / LIKELY FOLLOW-UP
Tie the answer to business numbers: "For payments I accept the latency for RPO zero; for analytics I use async."

Q23. What are replication slots, and how can they fill the disk?

SHORT ANSWER (say this first)
A replication slot makes the primary keep WAL until a consumer confirms it has received it. If the consumer stops, WAL piles up and can fill the disk.

THINK OF IT LIKE THIS
A bookmark telling the library "do not throw away any page after this one until I have read it". If the reader disappears, the library keeps every page forever and runs out of shelf space.

```
SELECT slot_name, slot_type, active,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS retained_wal
FROM pg_replication_slots ORDER BY pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn) DESC;
-- slot_x | logical | f | 412 GB      <- inactive slot retaining 412 GB of WAL
SELECT pg_drop_replication_slot('slot_x');   -- only after confirming nobody needs it
```

- **Prevent:** alert on retained WAL and on inactive slots; cap with `max_slot_wal_keep_size`; delete slots when decommissioning consumers (DMS tasks, subscribers, Debezium).
- On Aurora, watch `TransactionLogsDiskUsage` and `OldestReplicationSlotLag`.

INTERVIEW TIP / LIKELY FOLLOW-UP
"Disk filling up" is a favourite scenario question; slots are answer number one.

Q24. How do backups and point-in-time recovery (PITR) work in PostgreSQL?

SHORT ANSWER (say this first)
PITR combines a base backup (a full copy) with archived WAL. You restore the copy and replay WAL up to the exact moment before the problem.

THINK OF IT LIKE THIS
A photo of your house taken yesterday plus a CCTV recording since then. To see the house at 10:14, start from the photo and play the recording until 10:14.

1. Take a base backup regularly (`pg_basebackup`, or pgBackRest/Barman for incremental, compressed and verified backups).
2. Archive WAL continuously (`archive_mode = on` plus `archive_command` or `archive_library`).
3. To recover, restore the base backup into an empty data directory.
4. Set the recovery target and create the signal file, then start PostgreSQL. It replays WAL and stops at the target.
5. Check the data, then promote it to a normal server (`recovery_target_action = promote`).

```
# postgresql.conf (restored server)
restore_command         = 'cp /archive/%f %p'          # or pgbackrest --stanza=main archive-get %f
    "%p"
recovery_target_time    = '2026-10-27 10:14:00+00'
recovery_target_action  = 'promote'
# then:  touch $PGDATA/recovery.signal   and start the server
```

- `pg_dump` makes a **logical** backup: portable and good for one database or table, but no PITR and slow for very large databases.
- **A backup you have not restored is not a backup:** schedule restore tests and measure the time (your real RTO).
- On RDS/Aurora, automated backups and PITR are built in; a restore creates a **new** instance/cluster.

INTERVIEW TIP / LIKELY FOLLOW-UP
Always give RPO and RTO in the answer, and mention the monthly restore test.

Q25. How do you upgrade a PostgreSQL major version with minimal downtime?

SHORT ANSWER (say this first)
Use pg_upgrade for fast in-place upgrades, or logical replication to a new cluster for near-zero downtime. Dump and restore is simplest but slowest.

Method	How it works	Downtime	Risk / notes
pg_dump / restore	Export everything, import into new version	Hours for big data	Simple and clean; needs `ANALYZE` after.
pg_upgrade (`--link`)	Converts the catalog, reuses data files	Minutes	Run `--check` first; keep a backup; rebuild replicas; `ANALYZE` after.
Logical replication	Build new cluster, replicate, switch applications	Seconds to a minute	More moving parts: sequences, DDL, large objects.
Blue/Green (RDS/Aurora)	Managed staging copy kept in sync, then switchover	About a minute	Check supported versions and limitations.

1. Read release notes; check extension and driver compatibility.
2. Rehearse on a clone of production; compare plans and timings of your top queries.
3. Take a backup/snapshot. Upgrade. Run `ANALYZE` (statistics are not fully carried over).
4. Update extensions, compare `pg_stat_statements` before and after, keep the rollback path until a full business cycle passes.

INTERVIEW TIP / LIKELY FOLLOW-UP
"I rehearse on a clone and measure" is the sentence that separates experience from theory.

6. Memory, Configuration and Connections

Q26. Which memory parameters matter, and what is the danger of work_mem?

SHORT ANSWER (say this first)
`shared_buffers` is the shared cache, `work_mem` is private memory per sort/hash operation, and `maintenance_work_mem` is for maintenance. The danger is that work_mem is multiplied by operations and sessions.

Parameter	Plain meaning	Typical starting point
`shared_buffers`	PostgreSQL's own page cache, shared by all sessions	About 25% of RAM (self-managed)
`effective_cache_size`	A hint about how much cache (PostgreSQL + OS) exists. Allocates nothing.	About 50-75% of RAM
`work_mem`	Memory for **each** sort or hash step in a query	4-16 MB; raise per role/session only
`maintenance_work_mem`	Memory for VACUUM, CREATE INDEX	512 MB - 2 GB
`random_page_cost`	Planner cost of a random page read	1.1-1.5 on SSD

Worked example of the danger. `work_mem = 64MB`. A query has 3 sort/hash nodes. 100 queries run at the same time. Worst case: 100 x 3 x 64 MB = **19.2 GB** just for work memory, which can trigger an out-of-memory kill on a 32 GB server that also needs shared buffers and the OS cache. Parallel workers multiply it further.

```
ALTER ROLE report_user SET work_mem = '256MB';      -- only the reporting role gets more
SET LOCAL work_mem = '512MB';                       -- or only for one transaction
```

INTERVIEW TIP / LIKELY FOLLOW-UP
Say: "I raise work_mem only per role or session, and I confirm with `log_temp_files` or temp file statistics whether spills are really happening."

Q27. Why do you need connection pooling, and how does PgBouncer work?

SHORT ANSWER (say this first)
Every PostgreSQL connection is a process that uses memory, so thousands of connections hurt performance. A pooler keeps a small number of real connections and shares them among many clients.

THINK OF IT LIKE THIS
A bank with 1,000 customers but 20 tellers. Customers wait in a lobby (the pooler), and each one goes to a teller only for the actual transaction, so the tellers are always busy and never overwhelmed.

PgBouncer mode	Connection returned to the pool	Caveat
Session	When the client disconnects	Least efficient; most compatible.
Transaction	After each transaction	Most efficient; breaks session features: `SET`, temp tables, advisory locks, `LISTEN/NOTIFY`, (older versions) prepared statements.
Statement	After each statement	No multi-statement transactions.

- Size the pool from CPU and workload (a few times the vCPU count is a common start), not from the number of application threads.
- On AWS, **RDS Proxy** provides pooling plus faster failover for Lambda and microservices.

INTERVIEW TIP / LIKELY FOLLOW-UP
A follow-up is "What breaks in transaction pooling?" Know the list above.

7. Amazon RDS and Aurora PostgreSQL

Q28. How is RDS/Aurora PostgreSQL different from running PostgreSQL yourself?

SHORT ANSWER (say this first)
AWS manages the servers, storage, backups, patching and failover. You lose OS access and the true superuser, and you configure the database through parameter groups.

- No SSH or OS access. Use **Enhanced Monitoring**, CloudWatch and Performance Insights instead.
- The `postgres` superuser is replaced by `rds_superuser`; configuration goes through **parameter groups** (not `postgresql.conf`/`pg_hba.conf`).
- Automated backups with PITR; a restore always creates a **new** instance or cluster.
- Only AWS-supported extensions; some parameters are restricted.

INTERVIEW TIP / LIKELY FOLLOW-UP
Show you know the operational model: you still own indexing, vacuum tuning, schema design, security roles and cost.

Q29. How is Aurora PostgreSQL different from RDS for PostgreSQL?

SHORT ANSWER (say this first)
Aurora separates compute from a shared, distributed storage layer, which gives fast failover, very low replica lag and storage that grows automatically.

THINK OF IT LIKE THIS
RDS is a server with its own notebook (disk) that is copied to a second server. Aurora is several servers that all read and write the same shared notebook, which is kept in six copies in three data centres.

Feature	RDS for PostgreSQL	Aurora PostgreSQL
Storage	EBS volume you size (autoscaling available)	Shared cluster volume, grows automatically, 6 copies across 3 AZs
Read replicas	Up to 15 with asynchronous WAL replication	Up to 15 readers sharing storage; lag usually very small
Failover	Typically 1-2 minutes (Multi-AZ instance)	Typically about a minute or less; promotion tiers
Extras	-	Global Database, Serverless v2, Query Plan Management, cluster cache management
Temp files	On the instance storage	On **local instance storage**: watch `FreeLocalStorage`

INTERVIEW TIP / LIKELY FOLLOW-UP
Do not claim Aurora is always better. Say it fits high-availability and read-heavy workloads, and cost depends on I/O patterns.

Q30. How do you troubleshoot a performance problem on Aurora/RDS PostgreSQL?

1. **CloudWatch:** CPU, FreeableMemory, DatabaseConnections, read/write latency, replica lag, `FreeLocalStorage`.
2. **Performance Insights:** load versus vCPU count, then slice by wait event, SQL, user and host.
3. **Inside the database:** `pg_stat_activity` (who is running/blocked), `pg_stat_statements` (top SQL), `EXPLAIN (ANALYZE, BUFFERS)`.
4. **Enhanced Monitoring:** per-process OS view when CPU or memory looks odd.
5. **Decide:** SQL/index fix, pooling, move reads to readers, then scale the instance class as a last resort.

INTERVIEW TIP / LIKELY FOLLOW-UP
Reading wait events is the senior skill: `CPU`, `IO:DataFileRead`, `Lock:*`, `Client:ClientRead` each point to a different next step.

8. Scenario Questions (Model Answers)

Q31. Scenario: CPU on the primary is at 95%. What do you do?

1. **Scope it:** is it constant or a spike? Which instance? Did anything change (deployment, new report, traffic)?
2. **Find the consumer:** Performance Insights top SQL, or `pg_stat_activity` for active queries and `pg_stat_statements` for totals.
3. **Classify:** one bad query; an N+1 pattern (huge call count); too many parallel sessions; autovacuum; a plan regression after stats changed.
4. **Mitigate fast:** cancel runaway queries, run `ANALYZE` on a table with stale stats, add an index (concurrently), move reports to a replica.
5. **Fix properly:** query/index changes, pooling, plan management, and alarms. Scale up only if load is legitimate.

INTERVIEW TIP / LIKELY FOLLOW-UP
End with prevention (alarm, weekly top-SQL review) and a result: "CPU dropped from 95% to 30%".

Q32. Scenario: the database disk is filling quickly. Where do you look?

Suspect	How to check	Fix
Inactive replication slot retaining WAL	`pg_replication_slots` retained WAL; Aurora `TransactionLogsDiskUsage`	Restart consumer or drop slot
WAL archive failing	Archiver errors in log; `pg_stat_archiver`	Fix archive target/permissions
Bloat (vacuum blocked)	Dead tuples, table size growth, old transactions	Remove blocker, vacuum, repack
Temp files from big sorts	`pg_stat_database.temp_bytes`, `log_temp_files`	Tune query, work_mem per role, temp_file_limit
Unbounded data growth	Largest tables by `pg_total_relation_size`	Retention, partitioning, archiving
Logs growing	Log volume, verbose logging settings	Reduce logging, shorten retention

INTERVIEW TIP / LIKELY FOLLOW-UP
List suspects in order of likelihood and say which you would check first (slots).

Q33. Scenario: the application became slow right after a release. How do you find out why?

1. Compare `pg_stat_statements` and Performance Insights before and after: which statements changed time or call count?
2. Look for **new queries** from the release (missing indexes) and **N+1 patterns** (call count exploded).
3. Check locks: did a migration or long transaction block others? (`pg_blocking_pids`, `pg_stat_activity`).
4. Check plans for the changed queries and the freshness of statistics, especially after a big data change.
5. Roll back or feature-flag if impact is severe; fix indexes/queries; add a performance test to the release pipeline.

INTERVIEW TIP / LIKELY FOLLOW-UP
Mention communication: you give a status update and a rollback decision time, not just technical steps.

9. Glossary in Plain English

Term	Plain meaning
MVCC	Keeping multiple row versions so readers and writers do not block each other.
Tuple	One version of a row.
Dead tuple	An old row version no transaction needs any more, waiting to be cleaned.
Bloat	Wasted space in tables or indexes caused by dead rows.
VACUUM	Cleanup that makes space reusable and freezes old rows.
Freeze	Mark an old row as visible to everyone so transaction ID wraparound cannot hide it.
Wraparound	The 32-bit transaction counter running out of room; prevented by freezing.
WAL	Sequential log of changes written before the data files; basis of recovery and replication.
Checkpoint	Flush of all changed pages to disk, creating a recovery starting point.
Snapshot	The set of transactions a query considers committed when it starts.
Isolation level	Rules for what a transaction can see while others change data.
Replication slot	A bookmark that stops the primary deleting WAL a consumer still needs.
RPO / RTO	How much data you can lose / how long recovery may take.
PITR	Restoring a database to a chosen point in time using a backup plus WAL.
Partition pruning	The planner skipping partitions that cannot contain matching rows.
Statistics	Summaries of data distribution that the planner uses to choose plans.
EXPLAIN ANALYZE	Runs a query and shows the plan with real timings and row counts.
Connection pooler	A middle layer that shares few real connections among many clients.
Parameter group	AWS-managed set of database settings for RDS/Aurora.
Wait event	What a session is waiting for (CPU, I/O, a lock, the client).